Thinning-Stable Metaprogramming

András Kovács

University of Cambridge

13rd Oct 2026, IFIP WG 2.11 meeting, Niagara-on-the-Lake

Stability

In metaprogramming, it's nice if we don't have to explicitly handle object-language scopes.

```
f : Exp A -> Exp A
```

We'd like **f** to work on expressions containing arbitrary free variables.

At least, we'd like a closed such **f** to have the following semantics

```
⟦f⟧ : ∀ Γ. Exp Γ A -> Exp Γ A
```

Stability

Just having $[\![\texttt{f}]\!]$ `: ∀ Γ. Exp Γ A -> Exp Γ A` is not enough.

First, there's no *hygiene*. `f` could inspect `Γ` in arbitrary ways, and e.g. return different expressions in empty and non-empty scopes.

Stability

Second, There's not even an obvious way to have *binders*. With STLC as an object language, we'd like at least

```
lam : (Exp A -> Exp B) -> Exp (A ⇒ B)
```

However, the standard indexed-set semantics demands

```
⟦lam⟧ : ∀ Γ. (Exp Γ A -> Exp Γ B) -> Exp Γ (A ⇒ B)
```

where we can't conjure a fresh variable to apply the inner function to.

Explicit contexts

One approach: have explicit contexts in user-facing language.

```
f : ∀ Γ. Exp Γ A → Exp Γ A
```

For example: Moebius, Cocon, layered modal TT, typed FOAS.

- We can do *arbitrary* intensional analysis.
- A lot of bureaucracy; ergonomics is a challenge.

Stability under substitution

```
⟦f⟧ : ∀ Γ. Exp Γ A → Exp Γ A
  such that
∀ (Δ : Scope) (σ : Sub Δ Γ) (a : Exp Γ A). ⟦f⟧ (a[σ]) = (⟦f⟧ a)[σ]
```

A standard semantics for HOAS, logical frameworks & two-level calculi.

Barely any intensional analysis: no expression can be distinguished from a variable.

Easy faithful representation of object binders:

```
lam   : (Exp A -> Exp B) -> Exp (A ⇒ B)
⟦lam⟧ : ∀ Γ. (∀ Δ. Sub Δ Γ -> Exp Δ A → Exp Δ B) -> Exp Γ (A ⇒ B)
```

Stability under variable renaming

A renaming from **Γ** to **Δ** maps every **Δ**-variable to a **Γ**-variable.

To faithfully represent binders, we need an extra *type of variables*, and e.g. have

```
lam : (Var A -> Exp B) -> Exp (A ⇒ B)
```

Metaprograms can distinguish variables and non-variables, but e.g. can't decide equality of expressions...

... because two distinct variables can become equal by renaming.

Used in metatheory of TTs, but rarely in metaprogramming (?)

Stability under injective variable renaming

Equality of expressions can be decided.

But binders are much more fiddly: it's a *sub-structural* function type.

Example: *nominal sets*.

Example: the nullary parametric type theory (NPTT) of Muylder, Nuyts and Devriese[1].

[1] Muylder, Nuyts, Devriese: *Nominal Type Theory by Nullary Internal Parametricity*

Stability under thinning

A thinning from **Γ** to **Δ** deletes zero of more binders from **Γ**, resulting in **Δ**.

Contravariant action: if **σ : Thinning Γ Δ** and **e : Exp Δ A**, then **e[σ] : Exp Γ A**.

It's the *weakest* stability requirement that makes sense for hygienic metaprogramming.

ThinLF

A mostly hypothetical language. Agda/Idris syntax with extra nominal features. Based on NPTT by Muylder, Nuyts & Devriese.

```
data Ty : Type where
  ι   : Ty
  _⇒_ : Ty → Ty → Ty

data Exp : Ty → NomType where
  var : Var (Exp A) → Exp A
  lam : (@ (Exp A) → Exp B) → Exp (A ⇒ B)
  app : Exp (A ⇒ B) → Exp A → Exp B
```

ThinLF

We can declare an inductive type to be nominal, and use **Var** and the nominal function type in the definition.

$$\frac{\Gamma \text{ context} \quad \Gamma \vdash A : \text{NomType}}{\Gamma, x @ A \text{ context}} \qquad \frac{\Gamma \vdash A : \text{NomType} \quad \Gamma, x @ A \vdash B : \text{Type}}{\Gamma \vdash (x @ A) \to B : \text{Type}}$$

$$\frac{\Gamma, x @ A \vdash t : B}{\Gamma \vdash \lambda\, x.\, t : (x @ A) \to B} \qquad \frac{\Gamma \vdash t : (x @ A) \to B}{\Gamma, y @ A, \Delta \vdash t\ y : B[y/x]}$$

$$\frac{\Gamma \vdash A : \text{NomType}}{\Gamma \vdash \text{Var } A : \text{Type}} \qquad \frac{}{\Gamma, x @ A, \Delta \vdash \#x : \text{Var } A}$$

ThinLF

Ordinary HOAS/LF programming is still available:

```
lam' : (Exp A → Exp B) → Exp (A ⇒ B)
lam' f = lam λ x. f (var #x)
```

No flipping:

```
flip : (@ A → @ B → C) → (@ B → @ A → C)
flip = λ f b a. f a b   -- ill-typed
```

No duplication:

```
dup : (@ A → @ A → B) → @ A → B
dup = λ f a. f a a   -- ill-typed
```

Freshness

`Fresh A x`: the type of **`A`**-values that don't depend on **`x`**.

```
Fresh : {A : NomType} → Type → @ A → Type
fresh : ((x @ A) → Fresh B x) → B
<_,_> : B → (x @ A) → Fresh B x
 -- fresh and <_,_> are definitional inverses
```

We can match under a binder to extract a **`Bool`**:

```
extractBool : (@ A → Bool) → Bool
extractBool f = fresh λ x. case f x of
  True  → <True, x>
  False → <False, x>
```

Abstracting over free variables

Another primitive operation:

```
rebind : ((@ A → B) → @ A → C) → (@ A → B → C)
```

(I only show the non-dependently typed version here).

We can use it to match on **Either** under a binder:

```
split : (@ A → Either B C) → Either (@ A → B) (@ A → C)
split f = fresh λ x. case f x of
  Left b  → rebind (λ b x. <Left b, x>) x b
  Right c → rebind (λ c x. <Right c, x>) x c
```

Some useful definitions

```
forget : {A : NomType}{B : Type} → (x @ A) → Fresh B x → B
forget = rebind (λ fr _. fresh fr)

map : (B → C) → (x @ A) → Fresh B x → Fresh C x
map f = rebind (λ b x. <f (fresh b), x>)

bind : (x @ A) → Fresh B x → (B → Fresh C x) → Fresh C x
bind x b k = k (forget x b)

Fresh' : {A : NomType}{B : Type} → B → @ A → Type
Fresh' b x = (b' : Fresh B x) × (forget b' = b)
```

Structure abstraction principles (SAP)

Most SAP-s from Muylder et al. work fine:

```
(@ A → ⊤)              ≃ ⊤
(@ A → ⊥)              ≃ ⊥
(@ A → (B + C))        ≃ ((@ A → B) + (@ A → C))
(@ A → (B × C))        ≃ ((@ A → B) × (@ A → C))
(@ A → B → C)          ≃ ((@ A → B) → (@ A → C))
(@ A → List B)         ≃ List (@ A → B)
((x @ A) → f x = g x) ≃ (f = g)
((x @ A) → Fresh B x) ≃ B
```

This one requires name swapping, so we need to make it primitive (or generalize **`fresh`**):

```
((x @ A) → Fresh (B x) y) ≃ Fresh ((x @ A) → B x) y
```

Example: counting variables

```
SAP-Nat : (@ A → Nat) → Nat
SAP-Nat f = fresh λ x.
   let go : Nat → Fresh Nat x
       go Zero    = <Zero, x>
       go (Suc n) = map Suc x (go n)
   go (f x)

countVars : Exp A → Nat
countVars (var _)   = 1
countVars (app t u) = countVars t + countVars u
countVars (lam f)   = SAP-Nat (λ x. countVars (f x))
```

Example: recovering scoped syntax

```
Scope : Type
Scope = List ((A : Ty) × Var (Exp A))

inScope : (Γ : Scope) → {A : Ty} → Exp A → Type
inScope Γ (var {A} x) = (A, x) ∈ Γ
inScope Γ (app t u)   = inScope Γ t × inScope Γ u
inScope Γ (lam f)     = (x @ Exp A) → inScope (Cons (A, #x) Γ) (f x)
```

Note: **inScope** is decidable if we have decidable equality for **Var**.

Example: well-typed interpreter on scoped syntax

```
⟦_⟧ : Ty → Type
⟦ι⟧     = Nat
⟦A ⇒ B⟧ = ⟦A⟧ → ⟦B⟧

SAP-⟦⟧-Ty : (@ A → ⟦B⟧) → ⟦B⟧
SAP-⟦⟧-Ty = ...

⟦_⟧ : Scope → Type
⟦Nil          ⟧ = ⊤
⟦Cons (A, _) Γ⟧ = ⟦A⟧ × ⟦Γ⟧

⟦_⟧ : {Γ A} → (t : Exp A) → inScope Γ t → ⟦Γ⟧ → ⟦A⟧
⟦var x  ⟧ xsc        γ = lookup x xsc γ
⟦app t u⟧ (tsc, usc) γ = (⟦t⟧ tsc γ) (⟦u⟧ usc γ)
⟦lam f  ⟧ fsc        γ = SAP-⟦⟧-Ty (λ x α. ⟦ f x ⟧ (fsc x) (α, γ))
```

Extra assumptions on variables

The semantics supports

- Decidable ordering on **`Var`**.
- Decidable comparison of a raw variable and a **`Var`**.

If we have these, we can define

- Decidable ordering (and equality) for **`Exp`**.
- General variable renaming as a recursive operation on **`Exp`**.
- Substitution as a recursive operation on **`Exp`**.
- Strengthening on **`Exp`** w.r.t. a raw variable, returning **`Decide (Fresh (Exp A) x)`**.

I'm not sure about the exact specification of variable operations.

ThinLF compared to Nullary Parametric TT

ThinLF has a lot of overlap with NPTT. What are the differences?

- ThinLF supports *indexed nominal datatypes*, where the indexing types are ordinary ThinLF types. NPTT only supports untyped nominal types in a straightforward way, and trying to have first-class indexing types seems rather complicated.
- NPTT supports swapping raw variables in instantiations, ThinLF doesn't.
- ThinLF supports decidable ordering of variables, NPTT is only compatible with decidable equality.

Conclusions

- We can do a lot in ThinLF, but I don't necessarily know how to do it efficiently and concisely.
- I'm hoping that we can build nice machinery on top of the primitives.
- Features like *pattern matching under binders* could be elaborated to ThinLF.
- I'm not quite sure about the set of primitives, or even the basic setup. Can we use "observational" binder types instead of manual SAP plugging?